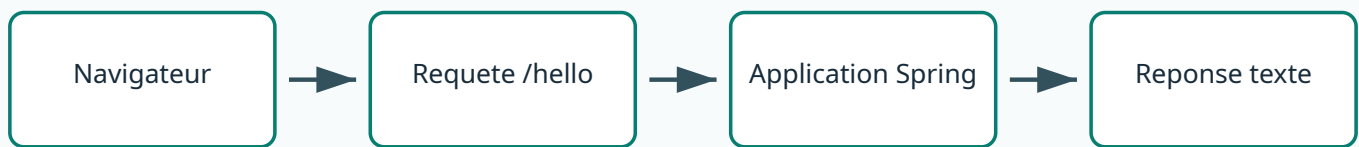


Cours 1 — Découvrir Spring et lancer sa première application

1. Partir d'un programme Java connu

Dans un programme console, `main` peut appeler `System.out.println("Bonjour")`, puis le programme se termine. Maintenant, nous voulons qu'un autre programme puisse demander ce message, éventuellement plusieurs fois. Notre application doit rester démarrée, attendre une demande et envoyer une réponse.

Le programme qui demande est le **client**. Celui qui attend et répond est le **serveur**. Un navigateur peut être un client. Les deux peuvent fonctionner sur le même ordinateur : c'est ce que nous ferons pendant les TP.



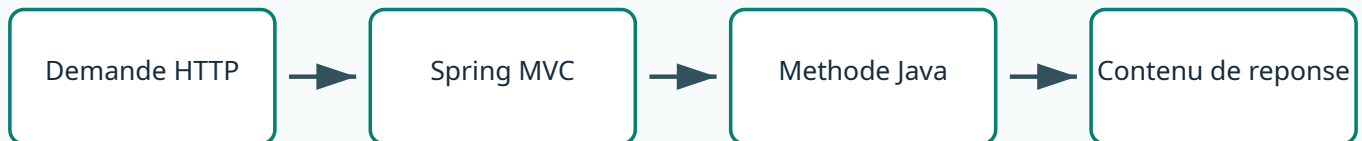
Une **API** est une interface par laquelle un programme utilise les fonctions d'un autre. Ici, cette interface sera accessible par des adresses HTTP. Nous détaillerons ce protocole à la séance 2.

2. Pourquoi utiliser un framework ?

Une bibliothèque contient du code réutilisable que l'on appelle. Un **framework** organise une partie du fonctionnement de l'application et appelle notre code aux endroits prévus. Il évite, par exemple, de programmer toute la réception des demandes réseau.

Spring fournit des outils pour construire des applications Java. **Spring Boot** simplifie leur préparation et leur démarrage. **Spring MVC** relie les demandes HTTP aux méthodes Java qui doivent répondre. Pour cette séance, retenir leurs rôles suffit ; il n'est pas demandé de connaître leur fonctionnement interne.

Notre responsabilité reste d'écrire la réponse et les règles de l'application. Le framework ne connaît pas spontanément le nom des utilisateurs ni les règles d'une boutique.



3. Lire le dossier d'un projet

Ouvrir `ateliers/01-demarrage`. C'est un petit projet complet, sans base de données ni mot de passe.

```
01-demarrage/  
├─ pom.xml  
└─ src/  
    ├─ main/java/fr/miage/debut/  
    │   ├─ Application.java  
    │   └─ HelloController.java  
    ├─ main/resources/application.properties  
    └─ test/java/...
```

`src/main/java` contient le code de l'application. `src/main/resources` contient sa configuration. Les fichiers de `src/test` vérifient le fonctionnement ; nous apprendrons à les lire à la séance 6.

Maven est l'outil qui télécharge les bibliothèques, compile et lance certaines commandes du projet. Sa recette s'appelle `pom.xml`. Une **dépendance Maven** est une bibliothèque nécessaire au projet. Le **starter web** est un ensemble de dépendances préparé pour une application web.

Dans le POM, repérer `spring-boot-starter-web`, `java.version` et `spring-boot-maven-plugin`. Le parent fixe des réglages communs ; le plugin permet notamment la commande de lancement. Il n'est pas demandé d'écrire tout le POM de mémoire.

4. Le point de départ reste `main`

```
package fr.miage.debut;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

`package` et `import` gardent leur sens habituel en Java. Le préfixe `@` indique une **annotation** : une information attachée à une classe ou une méthode, que des outils peuvent lire. Une annotation n'est pas un appel de méthode.

`@SpringBootApplication` désigne ici la classe de démarrage. `SpringApplication.run(...)` lance Spring et, avec le starter web, le serveur HTTP embarqué. « Embarqué » signifie qu'il démarre avec notre application : nous n'installons pas un serveur séparé.

Placer les autres classes dans `fr.miage.debut` ou ses sous-packages permet à Spring de les rechercher à partir de ce point de départ.

5. Une méthode qui répond

```
package fr.miage.debut;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Bonjour Spring";
    }
}
```

Un **contrôleur** est une classe qui reçoit des demandes web. `@RestController` signale ce rôle et indique que le résultat doit servir de contenu de réponse. `@GetMapping("/hello")` associe une demande GET sur `/hello` à cette méthode.

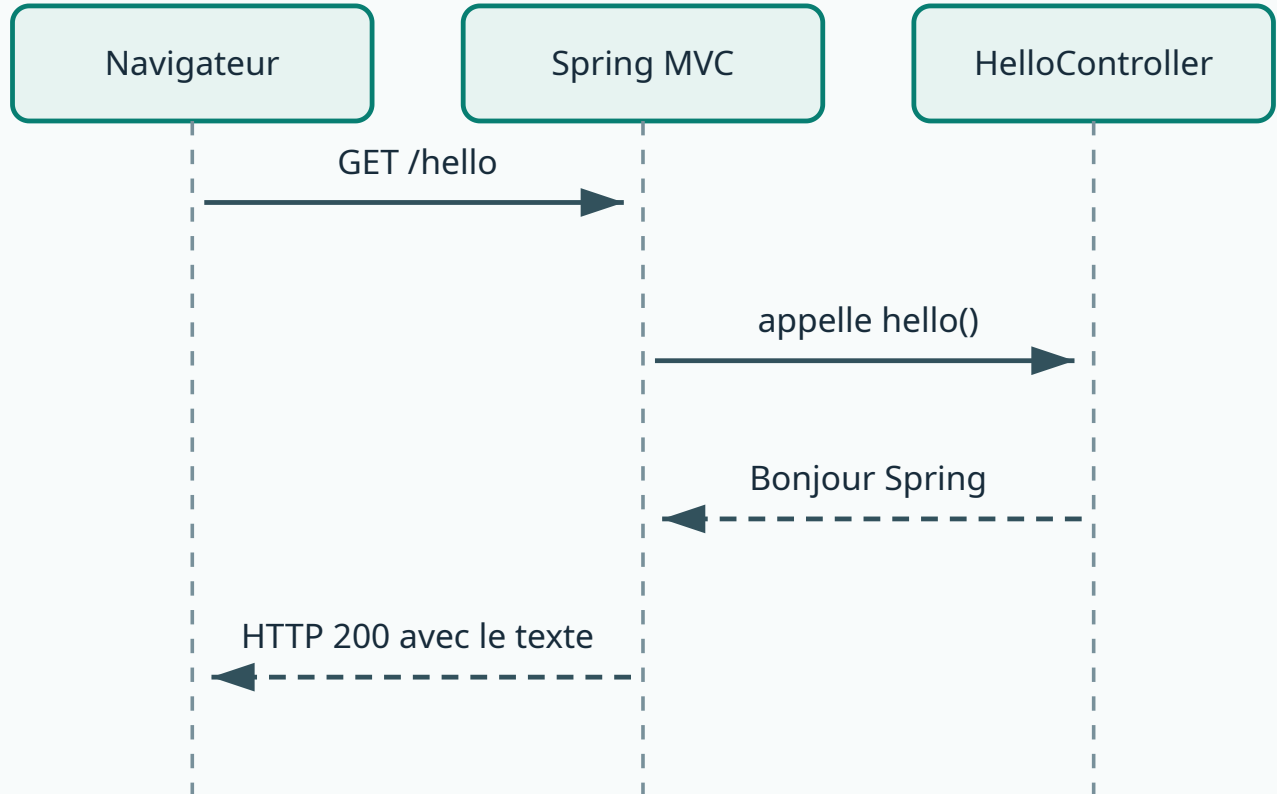


Diagramme de séquence UML

C'est Spring qui appelle `hello()` quand la demande arrive. Le `return` envoie le texte au client. Un `System.out.println` écrirait seulement dans le terminal du serveur : ce ne serait pas la réponse du navigateur.

6. Comprendre l'adresse

Dans `http://localhost:8080/hello`, `http` indique le protocole, `localhost` notre ordinateur, `8080` le **port** où l'application écoute, et `/hello` le chemin demandé. Un port permet de distinguer plusieurs services sur une même machine.

Si le serveur est arrêté, le navigateur ne peut pas se connecter. Si le serveur tourne mais que le chemin n'existe pas, il peut répondre avec une erreur 404. Nous apprendrons à lire les codes de réponse au cours suivant.

Démonstration — Lancement de Spring Boot et première réponse

Place dans le cours : cette démonstration illustre le concept présenté dans la partie précédente. Le [projet de démonstration du cours 1](#) permet de voir le comportement complet et les principaux composants. Elle sert d'exemple commenté ; les modifications sont réservées au TP.

La démonstration présente le démarrage d'une application Spring Boot puis une requête HTTP adressée à `/hello`. Spring initialise le contexte, démarre le serveur web embarqué et associe la route au contrôleur. La méthode Java retourne le texte qui devient le corps de la réponse HTTP.

Les classes principales sont `Application`, qui démarre l'application, et `HelloController`, qui expose la route. L'idée à retenir est qu'une application Spring fournit un serveur et relie les requêtes web aux méthodes Java.

TP guidé — obtenir puis modifier une réponse

Point de départ : atelier 01. Le poste doit être préparé avec la fiche de démarrage.

Étape 1 — vérifier les outils

Ouvrir un terminal dans `ateliers/01-demarrage`, le dossier contenant `pom.xml`. Exécuter `java -version` puis `mvn -v`. Vérifier que Maven utilise Java 25. Une commande introuvable se règle avec l'enseignant avant de modifier le code.

Étape 2 — démarrer

```
mvn spring-boot:run
```

Attendre le message `Started Application`. Le terminal reste occupé : l'application attend les demandes. Le premier lancement peut télécharger des dépendances et nécessite un accès réseau.

Étape 3 — appeler

Ouvrir `http://localhost:8080/hello` dans le navigateur. **Résultat attendu :** `Bonjour Spring`. Recharger la page : la méthode est appelée à nouveau. Lire les deux classes et tracer à la main le chemin du navigateur vers `hello()`.

Étape 4 — modifier

Remplacer le texte retourné par `Bonjour MIAGE`. Arrêter le serveur par Ctrl+C dans son terminal, puis relancer la même commande. Actualiser la page. Le projet ne recharge pas automatiquement les modifications.

Étape 5 — ajouter une route

Dans `HelloController`, ajouter une méthode `info()` répondant à `/info` par `Mon premier serveur Java`. Utiliser le même modèle que `hello()`. Tester les deux adresses puis essayer `/inconnu`.

À rendre : `HelloController.java` et trois phrases expliquant client, contrôleur et rôle du `return`. **Réussite :** deux chemins renvoient deux textes distincts ; l'étudiant sait arrêter le serveur.

Référence enseignant : première application Spring Boot.

Quiz — 8 questions

Une réponse attendue par question ; répondre avec les notions de cette séance.

1. Dans notre essai, qui est le client : le navigateur ou `HelloController` ?
2. Quel fichier décrit les dépendances Maven ?
3. À quoi sert Spring Boot dans notre projet ?
4. Quelle annotation signale la classe qui répond aux demandes web ?
5. Quel chemin faut-il demander pour appeler une méthode annotée `@GetMapping("/info")` ?
6. Le texte retourné par `return` et celui écrit par `System.out.println` arrivent-ils au même endroit ?
7. Pourquoi le programme reste-t-il actif après son démarrage ?
8. Après modification du code, quelles actions réalise-t-on dans ce TP avant de retester ?

► **Corrigé du quiz**